

VELLORE INSTITUTE OF TECHNOLOGY
VELLORE CAMPUS

MPMC LAB

Task 5 — Experiment Report

Registration Number : 24BCE0403
Name : Aryan Singh Sikarwar
Experiment Number : 5
Date : 01 April 2026
Status : HOSTELLER



Question 1

Write an Assembly Language Program (ALP) to execute basic arithmetic operations (Addition, Subtraction, Multiplication, and Division) on an ARM7 processor and verify the results using Keil μ Vision simulator.

AIM

To write, compile, and simulate an Assembly Language Program (ALP) for performing basic arithmetic operations — Addition, Subtraction, Multiplication, and Division — on an ARM7 processor using Keil μ Vision software, and to verify the outputs using the built-in simulator.

APPARATUS REQUIRED

- Keil μ Vision IDE (with ARM7 device support)
- Personal Computer / Laptop
- ARM7 Simulator (integrated in Keil μ Vision)

ALGORITHM

a) Addition

1. Start
2. Load the first operand (0x03) into register R0
3. Load the second operand (0x02) into register R1
4. Add R0 and R1; store the result in R2
5. Stop

b) Subtraction

1. Start
2. Load the first operand (0x06) into register R0
3. Load the second operand (0x02) into register R1
4. Subtract R1 from R0; store the result in R2
5. Stop

c) Multiplication

1. Start
2. Load the first operand (0x03) into register R0
3. Load the second operand (0x02) into register R1

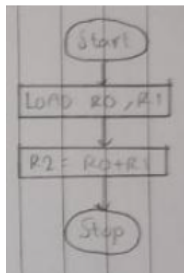
4. Multiply R0 and R1 using MUL; store the result in R3
5. Stop

d) Division

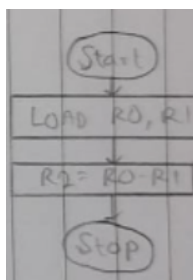
1. Start
2. Load dividend (0x05) into R1
3. Load divisor (0x02) into R2
4. Initialize quotient (R3) = 0
5. Compare R1 and R2
6. If $R1 < R2 \rightarrow$ Stop (division complete)
7. Else subtract R2 from R1
8. Increment quotient (R3)
9. Repeat from step 5
10. Stop

CODE

a) Addition



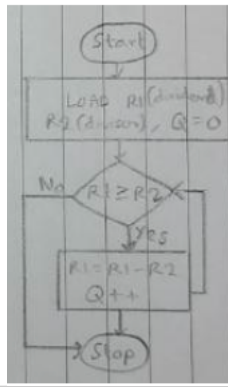
b) Subtraction



c) Multiplication



d) Division



RESULT

The Assembly Language Programs for Addition, Subtraction, Multiplication, and Division were successfully written, compiled, and simulated on the ARM7 processor using Keil μ Vision. The computed results were verified to be correct using the simulator's register watch window.

Question 2

Write an Assembly Language Program to generate a Sawtooth waveform using DAC interfaced with an ARM7 processor and verify on a processor board.

AIM

To write and execute an Assembly Language Program for generating a Sawtooth waveform by interfacing a DAC (Digital-to-Analog Converter) with an ARM7 processor, and to observe the waveform output on the processor board.

APPARATUS REQUIRED

- Keil μ Vision IDE
- ARM7 Processor Board (LPC2148 or equivalent)
- DAC Module (interfaced with ARM7)
- Oscilloscope (for waveform observation)
- Personal Computer / Laptop

ALGORITHM

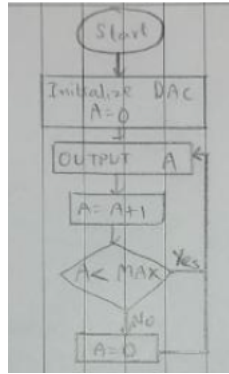
1. Start
2. Initialize the DAC and configure PINSEL register
3. Set the output value $A = 0$
4. Output value A to the DAC
5. Increment A by 1
6. If $A < \text{MAX value}$ $\rightarrow$ go to step 4
7. Else reset $A = 0$
8. Repeat from step 4 continuously

CODE

2)

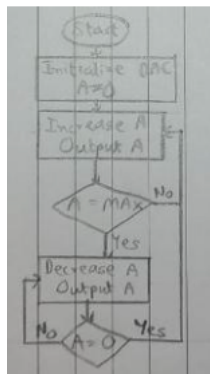
a) Sawtooth

1. Start
2. Initialize DAC
3. Set value = 0
4. Output value
5. Increment value
6. If value < max → repeat
7. Else reset to 0
8. Repeat



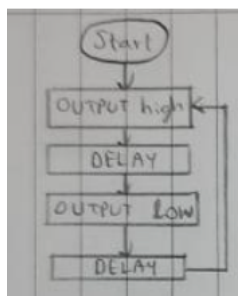
b) Triangle

1. Start
2. Initialize DAC
3. Increase value gradually
4. When max reached
5. Decrease value
6. When zero reached
7. Repeat



c) Square

1. Start
2. Initialize DAC
3. Increase value gradually
4. When max reached
5. Decrease value
6. When zero reached
7. Repeat



RESULT

The Assembly Language Program for generating a Sawtooth waveform was successfully written and executed on the ARM7 processor. The sawtooth pattern — linearly increasing from 0 to maximum and then resetting — was observed and verified on the processor board.

Question 3

Write an Assembly Language Program to generate a Triangular waveform using DAC interfaced with an ARM7 processor and verify on a processor board.

AIM

To write and execute an Assembly Language Program for generating a Triangular waveform by interfacing a DAC with an ARM7 processor, and to observe the output on the processor board.

APPARATUS REQUIRED

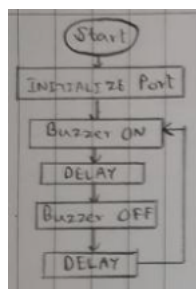
- Keil μ Vision IDE
- ARM7 Processor Board (LPC2148 or equivalent)
- DAC Module (interfaced with ARM7)
- Oscilloscope (for waveform observation)
- Personal Computer / Laptop

ALGORITHM

1. Start
2. Initialize DAC and configure PINSEL register
3. Set A = 0
4. (Rising Phase) Output A to DAC, increment A
5. If A < MAX $\rightarrow$ repeat step 4
6. (Falling Phase) When MAX is reached, decrement A, output A to DAC
7. If A > 0 $\rightarrow$ repeat step 6
8. When A = 0, repeat from step 4

CODE

1. Start
2. Configure port as output
3. Turn buzzer ON
4. Delay
5. Turn buzzer OFF
6. Delay
7. Repeat



1) Write ALP and to execute basic arithmetic operations in ARM7.

A: a) Addition: `area addition, code, readonly`
`mov r0, #0x03`
`mov r1, #0x02`
`add r2, r1, r0`
`l b l`
`END`

b) Subtraction: `area subtraction, code, readonly`
`mov r0, #0x06`
`mov r1, #0x02`
`sub r2, r0, r1`
`l b l`
`END`

c) Multiplication: `area multiplication, code, readonly`
`mov r0, #0x03`
`mov r1, #0x02`
`mul r2, r1, r0`
`l b l`
`END`

d) Division: `area division, code, readonly`
`mov r1, #0x05`
`mov r2, #0x02`
`li cmp r1, r2`
`bcc l`
`sub r1, r1, r2`
`add r3, #0x01`
`b li`
`l b l`
`END`

2) Generation of waveform using ARM processor

i) Sawtooth wave- `area DAK, CODE, READONLY`
`PINSEL1 EQU 0xE000C006`
`DACK EQU 0xE006C000`
`ldr v3, =0x000003FF`
`ldr v1, =PINSEL1`
`ldr v2, =0x00000000`
`str v2, [r1]`
`ldr v0, =DACK`
`ldr v2, =0x0`
`DtoA`
`ldr r1, r2, #6`
`str r1, [r0]`
`li add r2, r2, #1`
`cmp r2, r3`
`blt DtoA`
`ldr r2, =0x0`
`B DtoA`
`END`

ii) Triangular wave- `area trianglewave, CODE, READONLY`
`PINSEL1 EQU 0xE000C006`
`DACK EQU 0xE006C000`
`ldr v3, =0x000003FF`
`ldr v1, =PINSEL1`

`ldr v2, =0x00000000`
`str v2, [r1]`
`ldr v0, =DACK`
`ldr v2, =0x0`
`DtoAp`
`ldr r1, r2, #6`
`str r1, [r0]`
`add r1, r2, #1`
`cmp r2, r3`
`blt DtoAp`
`DtoAn`
`ldr r1, r2, #6`
`str r1, [r0]`
`sub r2, r2, #1`
`B DtoAn`
`B DtoAp`
`END`

iii) Square waveform using DAC of an ARM7 processor and verify in processor board -

`area square, code, readonly`
`io0dir equ 0xE0020008`
`io0data equ 0xE0020004`
`io0ctrl equ 0xE002000c`
`io0pin equ 0xE0020000`

`start`
`li r0, =io0dir`
`ldr r1, =0xffffffff`
`str r1, [r0]`
`ldr r2, =io0pin`
`str r1, [r2]`
`li delay, =0x00000000`
`ldr r3, =io0pin`
`str r1, [r3]`
`bl delay`
`b ctoad`
`delay`
`ldr r4, =0x0000ffff`
`l sub r4, #01`
`bne l`
`bv lr`
`done b done`
`end`

3)

Interfacing buzzer using ARM processor
Aim: To write ALP to interface buzzer with ARM7 processor
and make buzzer on and off with human perceivable delay

```
CODE: area buzzer, code, readonly
IOIDIR EQU 0xE0002018
IOISER EQU 0xB0028014
IOICLR EQU 0xB002801C
IOIPIN EQU 0xB0028010

START
LDR R0, IOIDIR
LDR R1, #0x00000000
STR R1, [R0]
LDR R1, IOISER
LDR R2, IOIPIN
STR R1, [R2]
```

```
BL DELAY
LDR R3, IOICLR
STR R1, [R3]
BL DELAY
B START
DELAY
LDR R4, #0x00F0000F
L SUBS R4, R4, #0x01
BNE L
BX LR
END
```

RESULT

The Assembly Language Program for generating a Triangular waveform was successfully written and executed on the ARM7 processor. The waveform — gradually rising to maximum and symmetrically falling back to zero — was observed and verified on the processor board.

Question 4

Write an Assembly Language Program to generate a Square waveform using DAC interfaced with an ARM7 processor and verify on a processor board.

AIM

To write and execute an Assembly Language Program for generating a Square waveform by interfacing a DAC with an ARM7 processor, and to observe the output on the processor board.

APPARATUS REQUIRED

- Keil µVision IDE
- ARM7 Processor Board (LPC2148 or equivalent)
- DAC Module (interfaced with ARM7)
- Oscilloscope (for waveform observation)
- Personal Computer / Laptop

ALGORITHM

1. Start
2. Initialize DAC, configure PINSEL and port registers
3. Output HIGH value (0xFFFFFFFF) to DAC
4. Wait for delay
5. Output LOW value (0x00) to DAC
6. Wait for delay
7. Repeat from step 3 continuously

CODE

```
        AREA mname, CODE, READONLY

IODIR   EQU 0x3F028004
IOSET   EQU 0x3F028014
IOCLR   EQU 0x3F02801C
IOPIN   EQU 0x3F028010

START

        LDR R0 = IODIR
        LDR R1 = 0xC2000000
        STR R1, (R0)
        LDR R3 = IOSET
```

```

        STR R1, (R3)

        BL  DELAY

        LDR R2 = IOCLR

        STR R1, (R2)

        BL  DELAY

        B   START

DELAY

        LDR R4 = 0XC0F0000F

L        SUBS R4, R4, #0x01

        BNE L

        BX  LR

        END

```

RESULT

The Assembly Language Program for generating a Square waveform was successfully written and executed on the ARM7 processor. The square wave output — alternating between HIGH and LOW states with a human-perceivable delay — was observed and verified on the processor board.

Question 5

Write an Assembly Language Program to interface a buzzer with an ARM7 processor and make the buzzer turn ON and OFF with a human-perceivable delay.

AIM

To write and execute an Assembly Language Program to interface a buzzer with the ARM7 processor such that the buzzer toggles ON and OFF with a human-perceivable delay, using GPIO port control registers.

APPARATUS REQUIRED

- Keil µVision IDE
- ARM7 Processor Board (LPC2148 or equivalent)
- Buzzer module (interfaced via GPIO port)
- Personal Computer / Laptop
- Connecting wires and power supply

ALGORITHM

1. Start
2. Configure the GPIO port as output using IODIR register
3. Turn buzzer ON by writing to IOSET register
4. Call DELAY subroutine
5. Turn buzzer OFF by writing to IOCLR register
6. Call DELAY subroutine
7. Repeat from step 3 continuously

CODE

```
CODE      AREA buzzer, CODE, READONLY

IODIR     EQU    0xE0028014

IOSET     EQU    0xE0028014

IOCLR     EQU    0xE002801C

IOPIN     EQU    0xE0028010

START

        LPR     R0 = IODIR

        LDR     R1 = 0xC2000000

        STR     R1, (R0)
```

```
        LDR  R3 = IOSET
        STR  R1, (R3)
        BL   DELAY
        LDR  R2 = IOCLR
        STR  R1, (R2)
        BL   DELAY
        B    START

DELAY
        LDR  R4 = 0XCOF0000F
L        SUBS R4, R4, #0x01
        BNE  L
        BX   LR
        END
```

RESULT

The Assembly Language Program for interfacing a buzzer with the ARM7 processor was successfully written, compiled, and executed. The buzzer toggled ON and OFF with a clear, human-perceivable delay, confirming correct GPIO control and interrupt-free delay loop operation.